Aviónica Software · Prueba de acceso 2026-2027

Faraday Rocketry UPV

Eden Rochman Sharabi

Octubre de 2026

Misión que he tomado como referencia

El enunciado no dice qué cohete hay que diseñar, así que para poder justificar las decisiones he supuesto un cohete sonda tipo EuRoC con un apogeo de entre 3 y 9 km. Lleva drogue en el apogeo, paracaídas principal a baja altura, aerofrenos movidos por un servo, radio de telemetría y GPS para encontrarlo al aterrizar. Como el objetivo del equipo es acabar llegando al espacio (más de 100 km), también lo he tenido en cuenta en algunas decisiones.

Para los cálculos he usado un empuje medio de unos 4000 N y una masa al despegue de unos 25 kg.

1. Ejercicio 1: Selección de microcontrolador y configuración básica

Apartado a

> *Selecciona un microcontrolador STM32 para la aviónica de vuelo y justifica tu elección (memoria, potencia de cálculo, consumo, periféricos, etc.).*

He elegido el **STM32F405RGT6**.

Descarté las familias sin FPU (F0, G0, L0 y F1) y comparé un micro de cada gama con sus datasheets: el F411, el F405 y el H743.

Tabla 1: Comparativa de los tres candidatos.

	STM32F411	STM32F405	STM32H743
Núcleo y reloj	Cortex-M4F, 100 MHz	Cortex-M4F, 168 MHz	Cortex-M7, 480 MHz
Flash / RAM	512 / 128 KB	1024 / 192 KB	2048 / 1056 KB
UART / CAN	3 / no	6 / 2	9 / 2
Consumo	≈10 mA	40 mA	71 mA (a 400 MHz)
Carga de CPU estimada	48 %	28 %	10 %

Para comparar la potencia de cálculo estimé cuánta CPU gastaría el software de vuelo: un filtro de Kalman de altitud a 1 kHz, un EKF de actitud a 500 Hz (pensando ya en la misión al espacio) y el control de los aerofrenos a 100 Hz, más una reserva para leer sensores, guardar datos y enviar telemetría. Para el coste de cada filtro usé la aproximación

$$N_{\text{operaciones}} \approx 6n^3 + 6n^2m + 4nm^2 + 2m^3,$$

donde n es el número de estados y m el de medidas. El cálculo está en la hoja «Carga CPU» de `avionica/comparativa_stm32.xlsx`.

Los tres cumplen, así que lo que decide son los periféricos. El F411 no tiene CAN y solo tiene 3 UART. El F405 tiene todo lo que necesito:

- 3 SPI y 3 I²C para los sensores.
- 6 UART para el GPS, la radio y la depuración.
- 2 CAN para hablar con otras placas.
- Timers con PWM para el servo.
- SDIO para guardar el vuelo en una tarjeta SD.
- 1 MB de flash y 192 KB de RAM.

Con todo eso la CPU queda a un 28 % de uso.

El H743 lo descarto porque se usaría al 10 %, consume más a su frecuencia máxima y es más complejo de programar (cachés, varios dominios de memoria y de alimentación). Para una misión al espacio podría tener sentido por su FPU de doble precisión.

Apartado b

> *Configura un proyecto básico en STM32CubeMX, que cuente con: al menos un bus SPI; al menos un bus I2C; la configuración necesaria para poder utilizar un programador ST-LINK.*

El proyecto está en `avionica/avionica_f405/`. Lo configuré así:

- **ST-LINK**: en SYS, *Serial Wire* (PA13 SWDIO y PA14 SWCLK).
- **Reloj**: cristal externo de 8 MHz en lugar del oscilador interno, que es menos preciso, con el PLL en $M = 4$, $N = 168$, $P = 2$:

  $$f = \frac{8\ \text{MHz}}{4} \cdot \frac{168}{2} = 168\ \text{MHz}.$$

  También activé el *Clock Security System*, para que el micro pase al reloj interno si el cristal falla.
- **SPI1** (PA5, PA6 y PA7) como maestro, con el CS por GPIO (PA4) para poder compartir el bus entre varios sensores.
- **I2C1** (PB6 y PB7) a 400 kHz para el barómetro.

El único problema fue de herramientas: CubeMX2 no soporta el F405 (solo la serie STM32C5), así que usé el CubeMX clásico (6.18).

El proyecto compila sin errores ni avisos.

2. Ejercicio 2: Selección e integración de sensores

Apartado a

> *Propón y justifica qué sensores consideras imprescindibles en la aviónica de un cohete sonda. Elige los modelos (ejemplo: acelerómetro HLIS200), justificando por qué.*

Lo que tiene que medir el cohete en cada fase:

- La aceleración, para detectar el despegue y el final del motor.
- El giro, para conocer su orientación.
- La presión, para detectar el apogeo y la altura del paracaídas principal.
- La posición, para encontrarlo al aterrizar.

De ahí salen cuatro sensores imprescindibles. Comparé varios modelos de cada uno en `seleccion_sensores.xlsx`.

Tabla 2: Sensores elegidos.

Sensor	Modelo	Por qué
IMU	BMI088	Aguanta vibraciones fuertes; ±24 g con un 57 % de margen
Acelerómetro de alta g	ADXL375	13 bits sobre ±200 g: mucha más resolución
Barómetro	MS5611	Mide hasta 10 hPa, unos 31 km de altitud
GPS	MAX-M10S	Consume unas 4 veces menos y es más pequeño

IMU: BMI088. Lo más importante es que el acelerómetro no se sature durante el motor, porque ese error se arrastra a la velocidad y la altura del resto del vuelo. La aceleración máxima con el motor encendido es:

$$a = \frac{T}{m} - g = \frac{4000}{25} - 9{,}81 \approx 150 \text{ m/s}^2 \approx 15{,}3\, g.$$

Y el pico del motor puede ser un 30–50 % mayor que la media. Por eso descarté el ICM-42688-P (±16 g) aunque es el que menos ruido tiene. Entre el BMI088 (±24 g) y el LSM6DSO32 (±32 g) me quedo con el BMI088 porque está diseñado para entornos con vibraciones fuertes, como las del motor y la estructura. Con ±24 g tengo un 57 % de margen, y los golpes más fuertes (paracaídas, aterrizaje) los cubre el acelerómetro de alta g. Su mayor consumo es poco frente al del micro y el GPS.

Acelerómetro de alta g: ADXL375. Mide cuando la IMU se satura, así que lo que importa es su resolución:

$$\text{H3LIS200DL: } \frac{200\, g}{2^8} = 0{,}78\, g \qquad \text{ADXL375: } \frac{400\, g}{2^{13}} = 0{,}049\, g$$

Con un error de medio paso, al integrar durante un segundo el H3LIS200DL acumularía hasta 3,8 m/s y el ADXL375 0,24 m/s. Elijo el ADXL375, que además muestrea hasta 3200 Hz.

Barómetro: MS5611. A 9 km la presión es de unos 308 hPa. El BMP390 mide hasta 300 hPa, así que llegaría con unos 150 m de margen y podría dejar de medir justo en el apogeo. El MS5611 llega a 10 hPa. Su precisión absoluta es peor (±1,5 hPa), pero la altura se calcula respecto a la presión en la rampa, así que ese error se cancela; su resolución es de unos 10 cm.

GPS: MAX-M10S. Los dos candidatos pierden la posición por encima de 4 g o de 500 m/s, es decir, durante el motor, y la recuperan en el descenso, que es cuando hace falta. La diferencia de precisión no importa para la recuperación. Decide el consumo: unos 7,6 mA frente a 28–36 mA del NEO-M9N.

Magnetómetro: no lo incluyo. Solo haría falta si el cohete corrigiera su orientación de forma activa, y además el metal y los cables del cohete perturban la medida.

Apartado b

> *Integra en el mismo proyecto generado en el anterior ejercicio el código necesario para inicializar y leer los datos de los sensores previamente seleccionados a través de los buses anteriormente configurados.*

Funcionamiento. Al encenderse, el micro comprueba que todos los sensores responden. Si alguno falla, lo reintenta tres veces reiniciándolo, y si sigue sin responder no entra en el bucle de vuelo (NO-GO). Lo aplico a todos los sensores, porque sin cualquiera de ellos se pierde la detección de eventos o la recuperación.

La comprobación de cada sensor:

- **BMI088**: registro de identificación `0x1E` (acelerómetro) y `0x0F` (giroscopio).
- **ADXL375**: registro de identificación `0xE5`.
- **MS5611**: respuesta por I²C y CRC de su memoria de calibración.
- **GPS**: una frase NMEA con checksum válido en menos de 2 s.

Conexiones. Los sensores inerciales van por el SPI1 y el barómetro por el I2C1. El GPS va por un UART, que es su interfaz habitual.

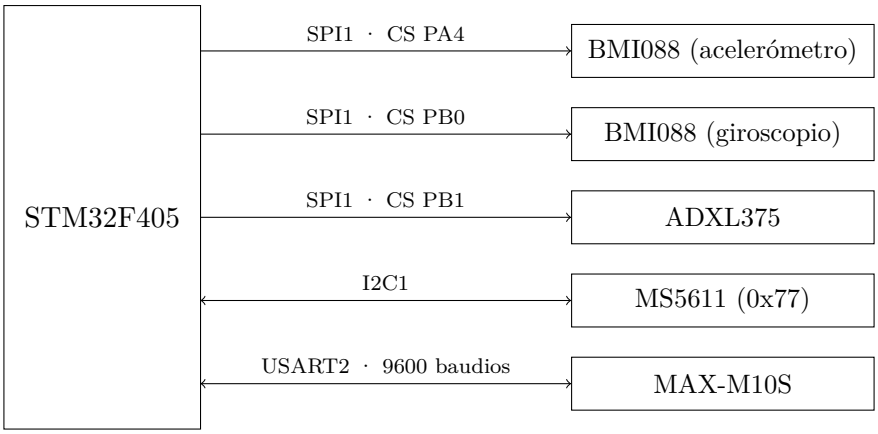


Figura 1: Cómo van conectados los sensores.

Problemas que encontré. El principal fue con el SPI: el ADXL375 solo funciona en modo 3 y como máximo a 5 MHz, y el bus estaba en modo 0 a 5,25 MHz. El BMI088 acepta los modos 0 y 3, así que el bus pasa a modo 3. Para la velocidad, el SPI1 parte de 84 MHz y el ADXL375 pide entre 2 y 5 MHz para usar sus 3200 Hz:

$$84/16 = 5{,}25 \text{ (se pasa)} \qquad 84/32 = 2{,}625 \text{ (vale)} \qquad 84/64 = 1{,}31 \text{ (no llega).}$$

Lo dejé en modo 3 con prescaler 32.

Otros problemas que salieron:

- El BMI088 tiene un CS para el acelerómetro y otro para el giroscopio, así que añadí dos CS (PB0 para el giroscopio y PB1 para el ADXL375).
- CubeMX pone el UART a 115200 baudios por defecto y el GPS arranca a 9600; lo cambié a 9600.
- El acelerómetro del BMI088 arranca en modo I²C y pasa a SPI con un flanco en su CS, así que hago una lectura de prueba al inicio. Además, en cada lectura envía primero un byte de relleno.
- El MS5611 tarda unos 9 ms por medida; para no bloquear el bucle (y perder bytes del GPS) lanzo la medida y la recojo 10 ms después.

El código. Un archivo por sensor (`bmi088`, `adxl375`, `ms5611` y `gps`) y, en `main.c`, código solo dentro de los bloques `USER CODE`:

```
/* Al arrancar: si algún sensor no responde, no se despega */
if (!comprobar(bmi088_init)) fallos_bit |= FALLO_BMI088;
if (!comprobar(adxl375_init)) fallos_bit |= FALLO_ADXL375;
if (!comprobar(ms5611_init)) fallos_bit |= FALLO_MS5611;
if (!comprobar(iniciar_gps)) fallos_bit |= FALLO_GPS;
if (fallos_bit != 0) { while (1) { } } /* NO-GO */

while (1) { /* GO: leer los sensores */
  bmi088_leer(datos.acc_g, datos.giro_dps);
  adxl375_leer(datos.alta_g);
  ms5611_actualizar(&datos.presion_hpa, &datos.temp_c);
  gps_actualizar(&datos.gps);
}
```

Las medidas se guardan en unidades físicas en una estructura global.

Verificación. El proyecto compila sin errores ni avisos. Sin la placa, probé en el ordenador la parte de cálculo (`avionica/test/test_drivers.c`):

- Las fórmulas del barómetro dan exactamente el ejemplo del datasheet: 1000,09 hPa y 20,07 °C.
- El CRC detecta un bit cambiado.
- El GPS extrae latitud, longitud y altura de una frase de ejemplo y descarta las de checksum incorrecto.

Lo que mejoraría.

- Probarlo con los sensores reales.
- Leer el GPS por DMA en lugar de por sondeo, dentro de FreeRTOS.
- Poner el GPS en modo vuelo con un comando de configuración de u-blox.
- Señalizar el NO-GO con un LED o por telemetría.

3. Ejercicio Personal

Detalla algún proyecto (o varios) de software / electrónica / hardware / antenas... que hayas realizado. Puedes incluir código, programas, fotos, vídeos, y todo lo que consideres necesario. Justifica para qué lo hiciste y qué conseguiste con ello.

Presento el proyecto del grupo de investigación en el que trabajo en la Universidad de Valencia. Está bajo un acuerdo de confidencialidad, así que lo describo a alto nivel.

Qué es. Un enjambre de drones para buscar personas perdidas en la montaña. Cada dron decide por sí mismo dónde buscar, a bordo y sin conexión a internet, y los drones se reparten la zona entre ellos. Hasta ahora el proyecto lo he desarrollado yo: la simulación, la integración con la controladora de vuelo, el modelo del dron real y las herramientas para lanzar y analizar los experimentos.

Con qué está hecho.

- **PX4** como software de vuelo y una controladora **Pixhawk 6C**.
- **ROS 2** para la lógica de cada dron y la comunicación entre ellos.
- **Gazebo** para simular el vuelo en modelos 3D de campus y de zonas de montaña.
- **QGroundControl** como estación de tierra.
- Un **visor 3D web** que he desarrollado para seguir las misiones en directo desde el navegador.

Empezó con un simulador 2D propio, pasó a la simulación completa con PX4 y después a la placa real en modo simulación, con la Pixhawk ejecutando el software de vuelo y sensores simulados.

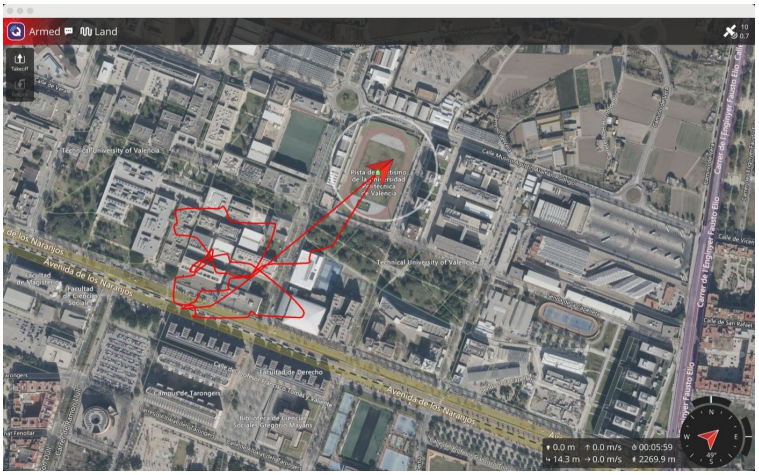


Figura 2: Trayectoria de una misión de búsqueda vista en QGroundControl, con la Pixhawk en modo simulación.

Principal problema. Al pasar al modelo de nuestro dron real, un Tarot Ironman 650, el dron se estrellaba en la simulación: PX4 venía configurado para un dron genérico, y la masa, el empuje y las ganancias del controlador no correspondían a nuestro dron, más grande y pesado. Construí un modelo del Tarot 650 a partir de su CAD y estoy ajustando las ganancias para él. Unas ganancias que funcionaban en la simulación por ordenador fallaron con la placa real por el retardo de los comandos, así que ahora lo incluyo en el ajuste.

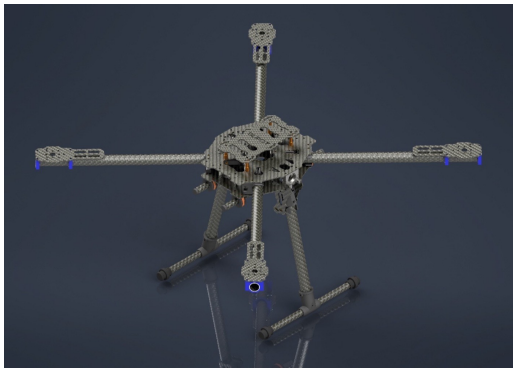


Figura 3: Modelo CAD del Tarot Ironman 650.

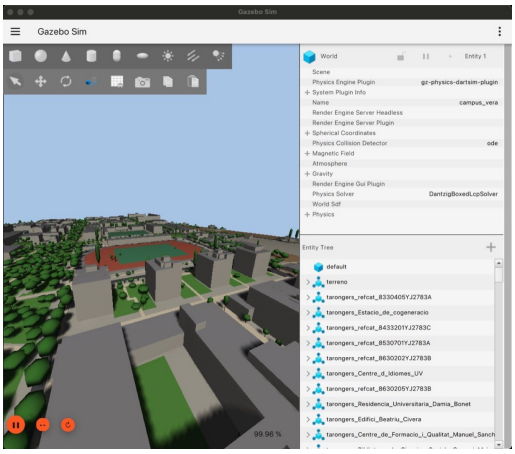


Figura 4: Modelo 3D de un campus en Gazebo.

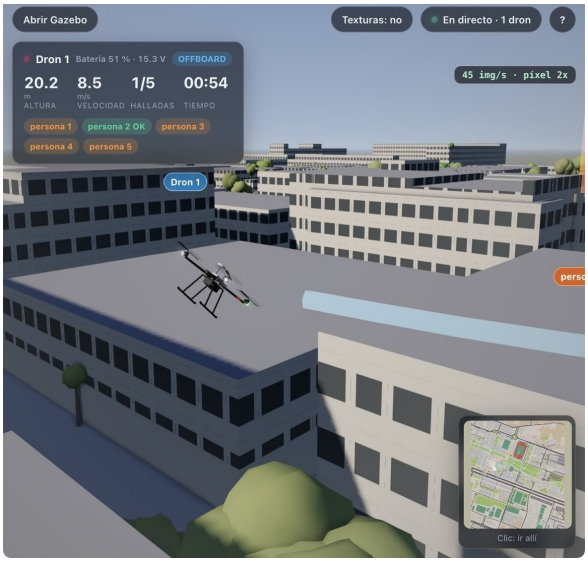


Figura 5: El visor 3D web que he desarrollado para seguir las misiones en directo: el dron simulado sobre el campus, con su altura, velocidad, batería y las personas encontradas.

Qué he aprendido. Drones, placas de vuelo, navegación por satélite y automatización de pruebas. Lo que más se relaciona con la aviónica de un cohete:

- La Pixhawk 6C lleva un STM32H743, el micro que comparé en el ejercicio 1.
- PX4 fusiona IMU, barómetro y GPS con un filtro de Kalman extendido.
- Comparé la precisión del GPS de la placa con y sin correcciones EGNOS.
- Probé la placa real con sensores simulados antes de volar.
- El ajuste del controlador es un problema parecido al del control de los aerofrenos.